[OPEN-SOURCE RESEARCH PROJECT]

AegisLEO

post-quantum satellite telemetry security
on home lab exercise

Jamie Grunewald · CypherCon 2026

Jamie Grunewald

// jgrunewald

VETERAN
U.S. Military — satellite comms are life-or-death, not academic

EDUCATION
M.S. Cybersecurity — Univ. of Delaware
NSA CAE-designated program

DAY JOB
Incident response · Threat Hunting
Full spectrum defense

ASPIRATION
PhD — Dakota State University
PQC · satellite comms · edge ML

// PROJECTS

AegisLEO ← you are here

Post-quantum satellite telemetry testbed
Pi 5 · Jetson Orin · LoRa · ML-KEM · autoencoder

Classical Cipher Cryptanalysis

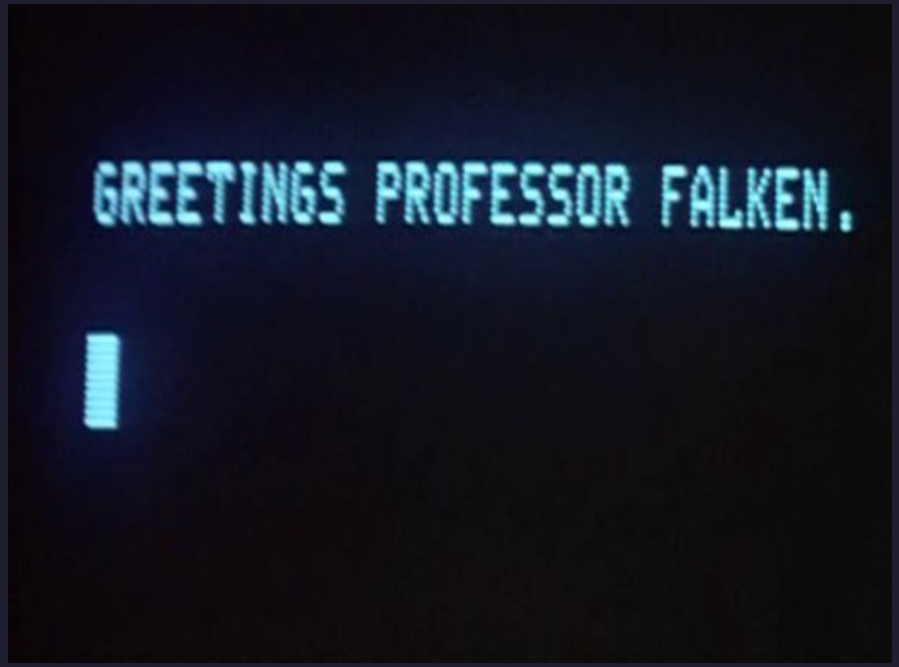
Shotgun hill-climbing · CypherCon 2025
github.com/JamieGrunewald/Crypto

Father and Husband

Curious 13-year-old son – Answering “why?” more than “what” – Encouraging curiosity

remember when the internet was fun?

- built by curious people — no budget, no permission
- BBS · phreaking · first web servers under a desk
- curiosity over permission — the original hacker ethic
- that energy built everything we rely on today
- my project exists because that energy never died, staying curious, never stop learning



the quantum threat is real

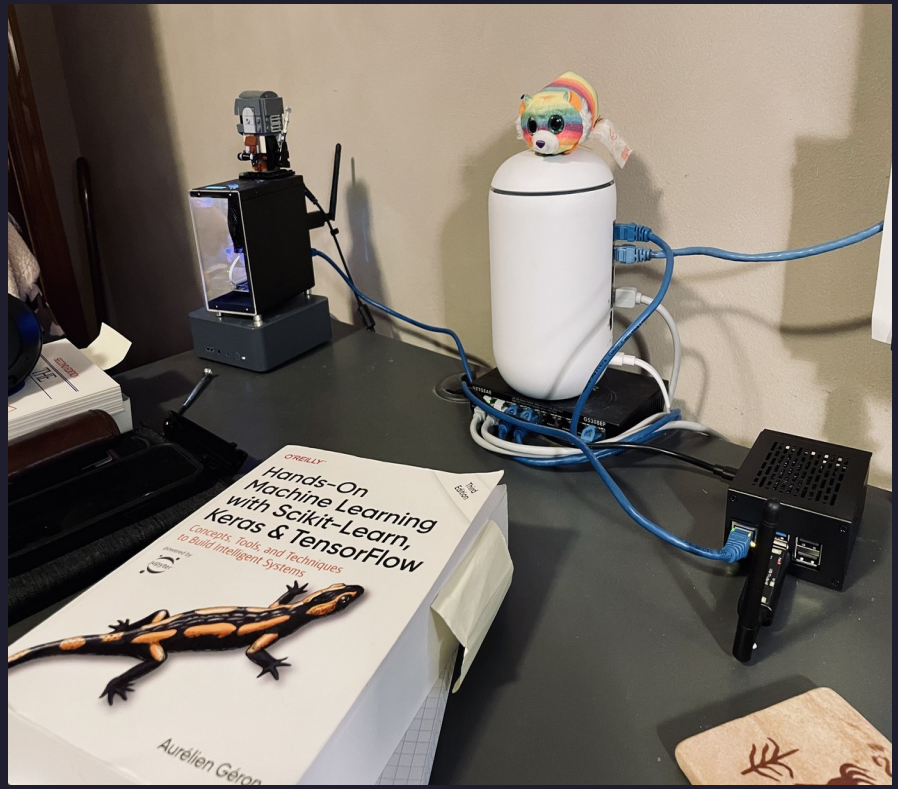
- Shor's algorithm breaks RSA + ECC — not weakens, breaks
- harvest now, decrypt later — recording traffic today
- GPS · weather · banking · military — all pre-quantum
- NIST PQC standards finalized 2024 — FIPS 203 · FIPS 204
- the question: is anyone deploying them?

CIA triad
+ one more

- Confidentiality — ML-KEM-1024 + AES-256-GCM
- harvest-now-decrypt-later: solved
- Integrity — ML-DSA-65 signatures
- forged packets: rejected at verification
- Availability — chunk/NACK self-healing
- 20% RF loss: automatically recovered
- + Authenticity — autoencoder anomaly detection
- valid crypto, wrong values: also caught

meet the hardware

- Raspberry Pi 5 — the satellite
- transmitter.py · ML-KEM · ML-DSA · AES-GCM
- NVIDIA Jetson Orin Nano Super — ground station
- receiver.py · Docker · PyTorch autoencoder
- Waveshare SX1262 LoRa — the RF link
- 868 MHz · USB serial · half-duplex
- Kali Linux on Proxmox — the adversary
- 192.168.60.172 · same VLAN · penetrated



what is LoRa and why does it matter?

- Long Range spread spectrum — designed for IoT, used by CubeSats
- 868 MHz · SF7 · BW125 · 10dBm · half-duplex
- TinyGS global network — real amateur satellites use this chipset
- slow and lossy — ~20% packet loss in near-field conditions
- constraint drove every design decision in AegisLEO

the RF problem that nearly killed the project

- modules 18 inches apart on my desk — nothing worked
- hours of debugging framing code · byte-stuffing · serial buffers
- read the SX1262 datasheet · wrote unit tests · added logging
- root cause: near-field RF saturation — signal too strong
- moved Pi across the room — everything worked

Docker on the Jetson

- CUDA-enabled container · full Python crypto stack
- bind-mounted repo — edit on laptop → container sees it instantly
- no rebuilds during live RF debugging sessions
- fix a bug · test within 30 seconds · iterate
- containers aren't just for production — they're for midnight debugging

CCSDS — the satellite communication standard

- Consultative Committee for Space Data Systems
- NASA · ESA · every major space agency uses it
- defines: Space Packet Protocol · telecommand · telemetry
- AegisLEO is CCSDS-inspired — not a full implementation
- real satellites use this — real satellites lack PQC

AegisLEO CCSDS-inspired elements

APID	Application Process Identifier — packet routing
SEQ CTR	Sequence counter — replay detection
TIMESTAMP	TAI-format mission elapsed time
FRAMING	Start/end markers · stuffed length field
CHUNKING	Transfer frame — split + reassemble

what CCSDS does NOT define

PQC	No ML-KEM · No ML-DSA · No post-quantum anything
ML	No behavioral anomaly detection layer

the KEM lockbox explained simply

- ground station sends an open padlock — no key needed to lock it
- satellite puts session key inside · snaps it shut · sends it back
- only ground station has the key — one way, no round trip
- session key encrypts all telemetry with AES-256-GCM
- a quantum computer cannot open this padlock

ML-KEM is not DHKE

- DHKE is a negotiation — both parties contribute, live round-trip
- ML-KEM is a lockbox — one-way encapsulation, no negotiation
- DHKE hardness: discrete log — Shor's breaks it
- ML-KEM hardness: Module LWE lattice — no quantum speedup
- ML-KEM's lock doesn't break when quantum arrives

DHKE — negotiation

FLOW: Alice sends g^a · Bob sends g^b · both compute g^ab

HARD PROB: discrete logarithm — broken by Shor's

ROUND TRIP: required — both sides must contribute

ML-KEM — lockbox

FLOW: receiver sends open padlock · sender wraps · done

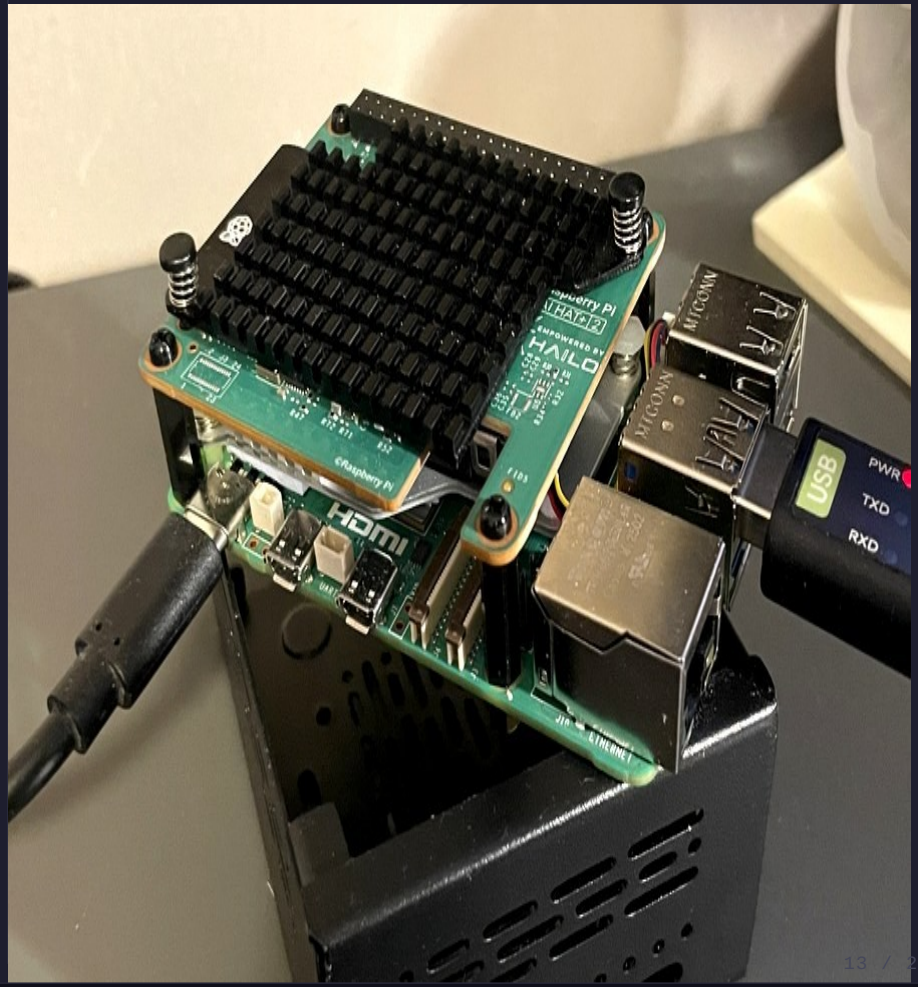
HARD PROB: Module Learning With Errors — no quantum speedup

ROUND TRIP: not required — one-way encapsulation

AegisLEO session-init: 6,716 bytes · 42 chunks · 7 NACK cycles · 5 minutes

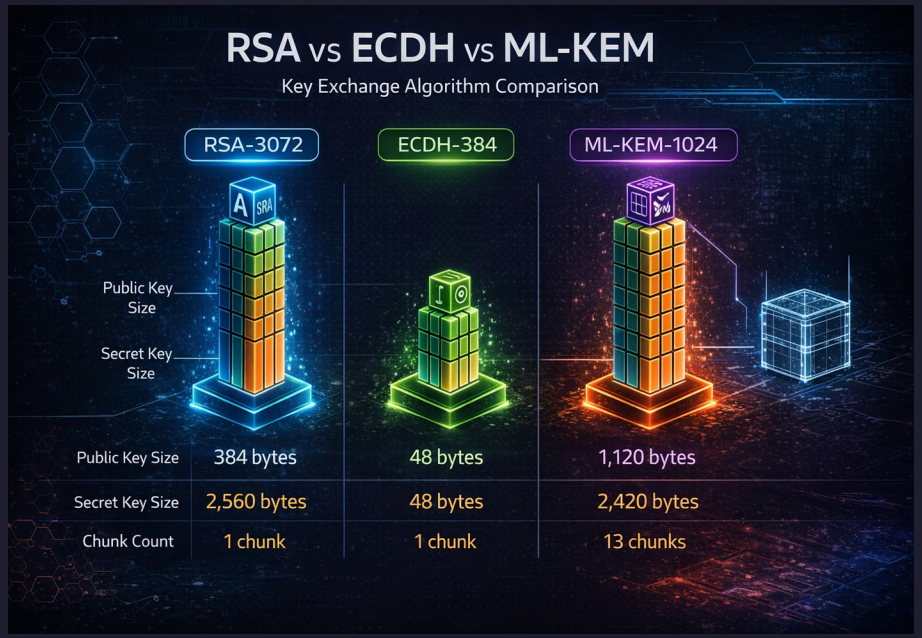
the full crypto stack

- ML-KEM-1024 — post-quantum key exchange
- FIPS 203 · Module Learning With Errors
- AES-256-GCM — authenticated payload encryption
- confidentiality + integrity in one operation
- ML-DSA-65 — post-quantum signatures
- FIPS 204 · every packet signed by satellite
- HKDF-SHA256 — key derivation from shared secret
- break one — the other two still hold



PQC is not free — it costs bytes

- RSA-2048 handshake: ~300 bytes — 2-3 LoRa chunks
- ECDH P-256: ~64 bytes key material
- ML-KEM-1024 ciphertext: 1,568 bytes
- ML-DSA-65 signature: 3,309 bytes
- AegisLEO session-init: 6,716 bytes · 42 chunks · 5 minutes



• raw=6627 · compressed=5035 · encoded=6716 · chunks=42

chunk assembly — the self-healing protocol

- 6,716-byte packet → 42 chunks of 160 bytes each
- each chunk: index · total · CRC32 · base64 payload
- receiver tracks missing chunks · sends NACK with list
- transmitter resends only missing — selective retransmit
- 35→29→23→17→11→7→2→0 — assembled

• REASSEMBLED · 7 NACK cycles · ~5 minutes

the bug that cost the most hours

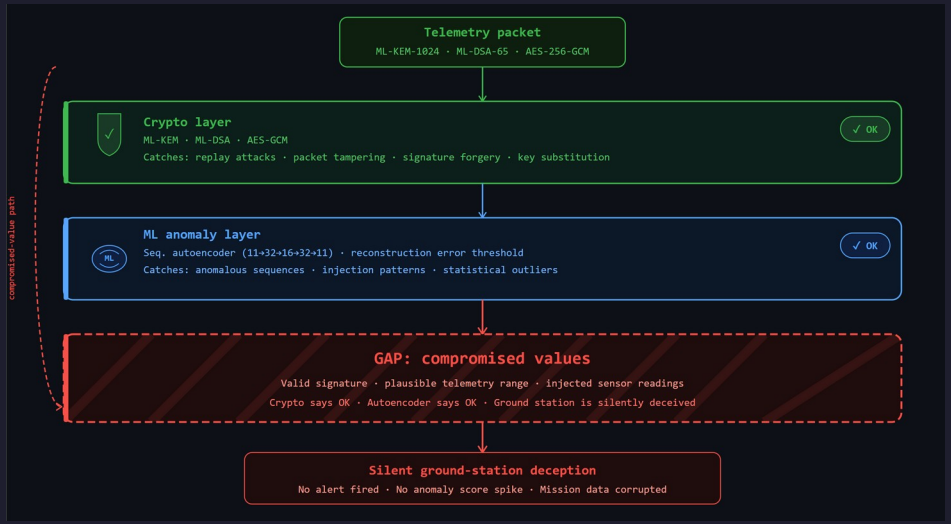
- chunks accumulate to 30 · then: reset to zero · repeat
- read every line of reassembly code · checked NACK logic
- root cause: two TTL constants — receiver.py vs reassembly.py
- receiver.py: 120 seconds · reassembly.py: 25 seconds
- change 25.0 → 600.0 · problem solved · hours lost

```
#   message_id : integer sequence number, or None for session-init
#
# Using a tuple as a dict key works because tuples are hashable in Python.
# ============================================================================

# TTL constants — how long we wait before giving up on an incomplete message
#
# Session-init gets more time because it's large (31 chunks) and we really
# need it to succeed before any telemetry can flow.
#
# Telemetry gets less time because packets are smaller and we'd rather
# move on than hold stale state forever.

SESSION_INIT_TTL_SECONDS = 600.0
TELEMETRY_TTL_SECONDS    = 30.0

# The receiver's NACK packet can only list this many missing chunk indexes.
# If more are missing, we send the first N and wait for the next round.
MAX_MISSING_PER_NACK = 36


# Type alias for the reassembly key.
# Using a type alias makes the code easier to read — instead of writing
# tuple[str, str, int | None] everywhere, we write ReassemblyKey.
#
# str  = chunk_type  ("si" or "tc")
# str  = session_id  (hex string)
# int | None = message_id (sequence number, or None for session-init)
ReassemblyKey = tuple[str, str, int | None]


class ReassemblyFactory:
    """
    Manages all in-flight chunk reassembly buffers.

    One instance of this lives in receiver.py and handles every
    incoming message across all active sessions simultaneously.
```


• FIXED · SESSION_INIT_TTL_SECONDS = 600.0

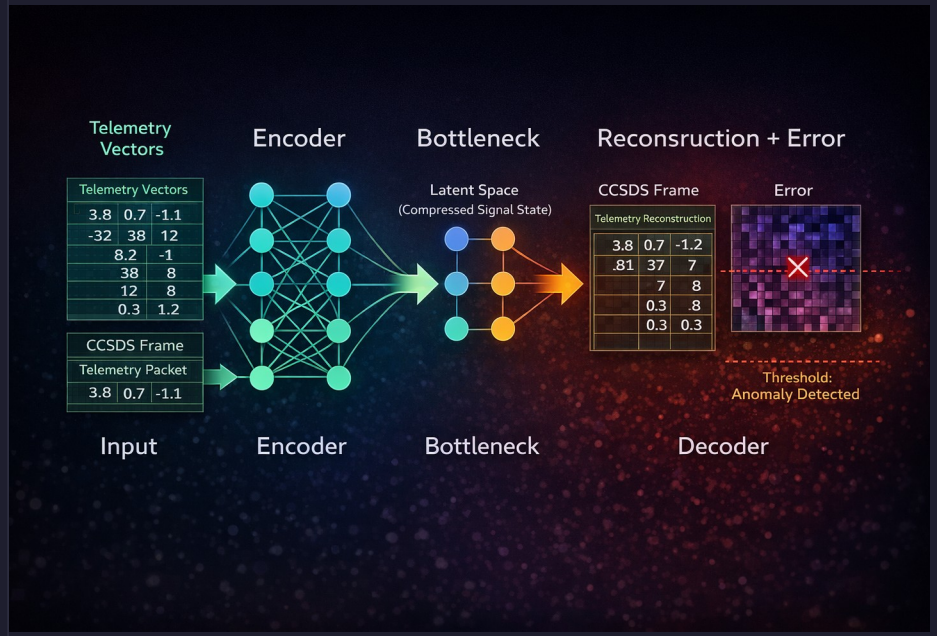
why a second defense layer?

- crypto catches: forgery and tampering
- crypto does NOT catch: compromised sensor firmware
- valid signature · valid encryption · wrong values
- what if future crypto is broken? need independent defense
- behavioral question: does this look like a real satellite?



what is an autoencoder?

- neural network: compress input → reconstruct input
- learns what normal looks like from training data
- reconstruction error = how well it remembers
- high error = input doesn't match learned normal pattern
- like showing a doctor, 2000 healthy EKGs then handing them one from cardiac arrest



• MODEL: seq_autoencoder.pt 11→32→16→32→11

training the model

- 11 features: temp · battery · lat · lon · alt · bus_v · bus_i · mode
- architecture: 11→32→16→32→11
- 2,000 samples · 20 epochs · ~30 seconds on CPU
- training data derived from real hardware observations — not textbook values
- threshold = mean + 3σ = 5.27 — automatically tuned

train_seq_autoencoder.py — orin-ground

```
Using device: cpu  (CUDA driver update pending)
Loading dataset...
Dataset windows shape: (2007, 4, 11)

Model architecture: SeqAutoencoder
  encoder: 11 → 32 → 16    decoder: 16 → 32 → 11

Starting training...

Epoch   1/100 → loss=6851661.3668
Epoch   2/100 → loss=   3500.8887
Epoch   3/100 → loss=   1248.7027
Epoch   5/100 → loss=    560.8341
Epoch  10/100 → loss=    590.1442
Epoch  20/100 → loss=    532.0423
Epoch  30/100 → loss=    600.1556
Epoch  42/100 → loss=    392.8992
Epoch  58/100 → loss=    337.7797
Epoch  64/100 → loss=    282.7476
Epoch  70/100 → loss=    193.8652
Epoch  73/100 → loss=    117.7514
Epoch  78/100 → loss=     88.7615
Epoch  83/100 → loss=     34.1152
Epoch  87/100 → loss=     37.6329
Epoch  94/100 → loss=     31.2163
Epoch  95/100 → loss=     23.4228
Epoch  98/100 → loss=     31.4875
Epoch  99/100 → loss=     33.3784
Epoch 100/100 → loss=    372.7383

Computing anomaly threshold...
Threshold calculated: 8.834264

Training complete.
Model saved     → models/seq_autoencoder.pt
Threshold saved → models/seq_threshold.json
```


• threshold: 5.27 · window_size: 4 · 1,996 training windows

the score separation

- normal telemetry: scores 0.9 — 3.0 · anomalous: False
- Kali injection seq=6: score 216 · ANOMALY DETECTED
- Kali injection seq=7: score 432 · ANOMALY DETECTED
- Kali injection seq=9: score 863 · ANOMALY DETECTED
- threshold: 5.27 · peak: 863 — 163× overage

```
ml_demo.py — orin-ground

── normal telemetry (──────────────────────────────

seq=1 mode=SUNPOINT  score=0.9412  threshold=5.27  ANOMALY: False
seq=2 mode=SUNPOINT  score=1.3081  threshold=5.27  ANOMALY: False
seq=3 mode=NOMINAL   score=2.0350  threshold=5.27  ANOMALY: False
seq=4 mode=SUNPOINT  score=2.9755  threshold=5.27  ANOMALY: False
seq=5 mode=NOMINAL   score=1.6829  threshold=5.27  ANOMALY: False

── Kali injection active · darpa-01 · 192.168.60.172 ────

seq=6 mode=NOMINAL   score= 216.4431  threshold=5.27
      temp=85.0C  battery=12%  bus_v=2.1V
      *** ANOMALY DETECTED ***  (41× overage)

seq=7 mode=NOMINAL   score= 432.1182  threshold=5.27
      temp=85.0C  battery=12%  bus_v=2.1V
      *** ANOMALY DETECTED ***  (82× overage)

seq=9 mode=NOMINAL   score= 863.7711  threshold=5.27
      temp=85.0C  battery=12%  bus_v=2.1V
      *** ANOMALY DETECTED ***  (163× overage)

──────────────────────────────────────────────────────

normal scores  : 0.94 — 2.97   all below threshold
anomaly scores : 216  →  432  →  863
threshold      : 5.27
peak score     : 863.7711  —  163× overage

▲  ML ANOMALY DETECTED · score=863.7711 · threshold=5.27
```

the adversary scenario

- Kali on Proxmox VM 100 · darpa-01 · 192.168.60.172
- same VLAN as Orin ground station — network penetrated
- session ID visible in plaintext chunk headers — Kali can sniff it
- Kali cannot forge ML-DSA-65 signature — no private key
- two outcomes: crypto catches it · ML catches it

defense layer 1 — crypto catches the forgery

- packet arrives with correct session ID — framing valid
- receiver attempts ML-DSA-65 signature verification
- signature: 32 bytes of 0xFF — garbage
- REJECTED: telemetry signature invalid
- cannot forge ML-DSA-65 without satellite private key

receiver.log — ground station

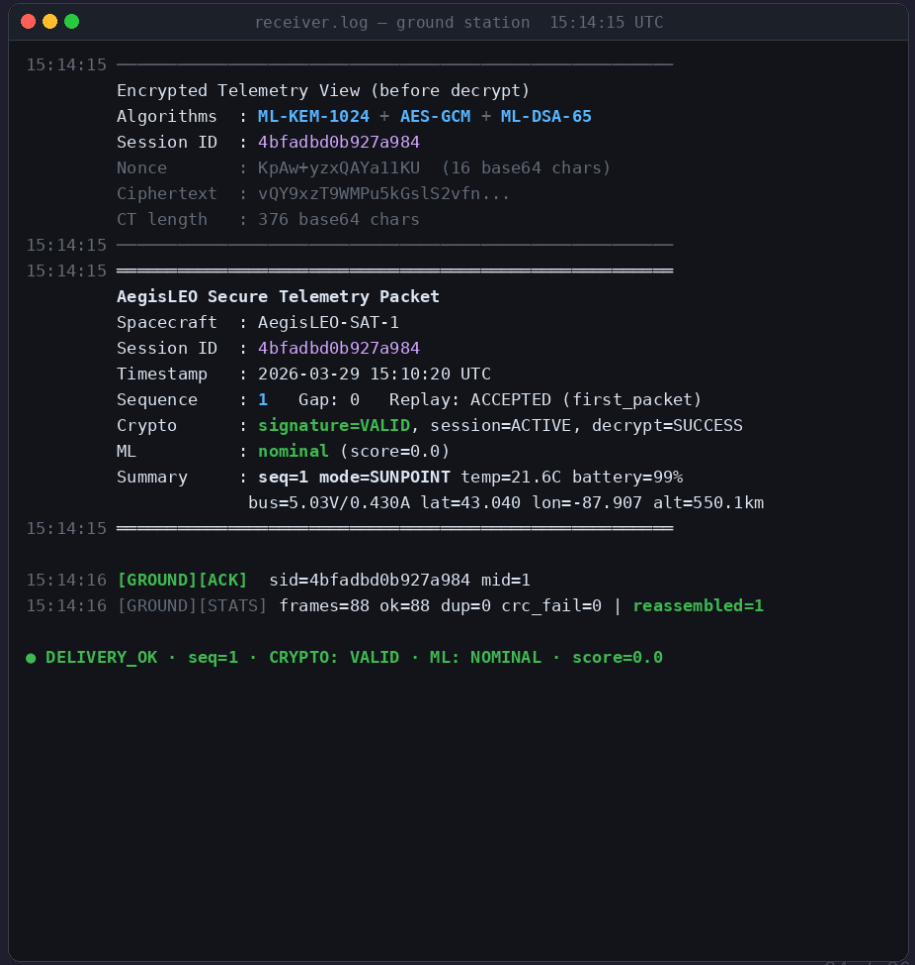
```
── Kali injection attempt ──────────────────────────────────────

16:22:37 [GROUND][CHUNK]       t=tc sid=4bfadbd0b927a984 mid=14 idx=42/45 missing=1
16:22:40 [GROUND][CHUNK]       t=tc sid=4bfadbd0b927a984 mid=14 idx=44/45
16:22:40 [GROUND][REASSEMBLED] t=tc sid=4bfadbd0b927a984 mid=14 len=5016

        ML-DSA-65 signature verification...
        sig bytes : FF FF FF FF FF FF FF FF ... (32 × 0xFF)
        expected  : valid ML-DSA-65 lattice signature
        result    : VERIFICATION FAILED

16:22:40 [GROUND] REJECTED: telemetry signature invalid

        crypto layer blocked the injection
        Kali has no ML-DSA-65 private key

── ML-DSA-65 · FIPS 204 · lattice-based signatures ─────────────
```

defense layer 2 — ML catches the behavior

- ML detector is independent of the crypto layer
- temp=85C · battery=12% · bus_v=2.1V — never in training data
- autoencoder tries to reconstruct — fails catastrophically
- score: 863 · threshold: 5.27 → ANOMALY DETECTED
- even with stolen keys — ML still fires on anomalous values

live telemetry — it actually works

- session 4bfadbd0b927a984 · seq=1 · 15:14 CST
- Crypto: signature=VALID · session=ACTIVE · decrypt=SUCCESS
- ML: nominal · score=2.97 · threshold=5.27
- lat=43.040 · lon=-87.907 · alt=550.1km — Milwaukee, LEO
- 12 of 16 packets delivered · 94 minutes of operation



● DELIVERY_OK · seq=1 · CRYPTO: VALID · ML: NOMINAL · score=2.97

what's next

- Grafana dashboard — live telemetry + ML score visualization
- Self-aware satellite — Hailo-10H AI HAT+ 2 on the Pi
- on-board LLM health reasoning before downlink
- Data fusion — unified crypto + ML + replay threat score

start building

[github.com/JamieGrunewald/AegisLEO]

Jamie Grunewald · CypherCon 2026

M.S. Cybersecurity · University of Delaware

"the internet was fun when people-built things"